

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Интеллектуальный анализ данных»

Тема: «Деревья решений»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Цель работы: на практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Ход работы

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
4. Оценить точность каждой модели на тестовой выборке;
5. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 9

- Glass Identification
 - Классифицировать тип стекла на основе его химического состава
 - **Задания:**
 1. Загрузите данные и стандартизируйте их;
 2. Разделите выборку;
 3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
 4. Сравните производительность моделей с помощью `classification_report (sklearn.metrics)`;
5. Укажите, какой класс модели определяют хуже всего, и предположите, почему.

Код программы:

```
from pathlib import Path
import argparse, base64, csv, hashlib, io, json, platform, time, zlib
import numpy as np
import sklearn
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.base import clone
from sklearn.preprocessing import StandardScaler
import catboost, xgboost
from catboost import CatBoostClassifier
from xgboost import XGBClassifier

DATA = 'c$|G!+iv4P2z~Eo_)Q{F7iPxau~OfOzq$twWN)DJs(-
&@z=;hfd8=VKz~MkTd>_9K=jnX>c) $J`#z8*+oqqGz*YUb_%IE6*d$^pA*Pr2qrZA?7#xUh^isSI>c>Vl73@>?_#{{gkzmDCW
jJ<Sw=yuxKsauOfq5hpZe&}fZPlJ->ecGdBhn>4EG76)CttHs372q_*FWxTbVQWon4F<c73qmw~)-kV@XViZ(_RTEP9w+q-
%vZASo8TvlpXGhD@2jgvc<*xpualq<;%m0Eb{x{3*<GJe`!s4{dw!xiFd?3F0s2C`$373F4C?myTuG*@2&PTimEDc`z6D{g-
{f8k>t5{}}?6k$8z5@x^Gu+b^rSnygj%K5a+Xg_@*=ZebX)3MWy(3lE{<Tax^ZQ=P>+$k^diy+oUWS*VO=XpiamuVR@0D?q(v+
DH6Suhf_VR>WTvaEb$iy8;t!3|A%7|QVZ_&mTzFU$p^^IF)tDXl#K(AEhXJeLAEdsWzbg;OzZA%DR0-
%|+zJ%&4TwBnyl|kz4&vGcmpTppbiBctwTZ$p&12xW(iy9m1vEi8~28CL=#W6S0OHhbQ2v*t$oJ|FX{FBDIUR`)t()Hbh?hl
~1<Qd{dJR#wI;)Rs8|m4`*+qJ6O<aC7b@e5ZdU75JsiGMq+1Ck7Z&g`9E|S4MautPiV+{;hkfxi&`6#(5V8cbURAep#Qg7R80
ePrlj*~TcAU$Hy1KyA%;D!Oh)*B{EhoTrL@28K`+u^PT8Tq{PRi=yh#D$ $l1~-
qyR5s%yj%vK^d%R_$au@e_az;rdR85rRDolvYRXL7smth+h`L-
41%+=Wa%|~W<s&h3ja=u!h&H@>o91f_dw>BT_&kPmH=@`pey+wQ98H$%CRWSra|P_>+Wtd&00luF{1Cr_Y-B(Du6Po-
dsPeMTL?pfN8)M=_zh*s`bf2qg+{;_iE!@s+nRuS^G1UFe~#JYG%!a_-MLcmixf-qT4|-
```

```
b5`wIZ6rRok_~ApK;_6aMI6o=lfj->^`TsAoUifORpBjS#Q~M>6k^%ppHEhUXGFHo=b|;nX-YWEp!S(#f@SJiteNDE'  
SHA256 = '119e30db8a4ea8b33723603743591a5f8229684e6236d89ef1966a72d7293607'
```

```
FEATURES = ['RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca', 'Ba', 'Fe']
```

```
GLASS_NAMES = {  
    1: 'building_windows_float',  
    2: 'building_windows_non_float',  
    3: 'vehicle_windows_float',  
    5: 'containers',  
    6: 'tableware',  
    7: 'headlamps'  
}
```

```
NAMES = ['Дерево', 'Случайный лес', 'AdaBoost', 'CatBoost', 'XGBoost']
```

```
GLASS_URL = 'https://archive.ics.uci.edu/ml/machine-learning-databases/glass/glass.data'
```

```
def load_data(path=None):  
    if path:  
        raw = Path(path).read_bytes()  
        digest = hashlib.sha256(raw).hexdigest()  
    else:  
        import urllib.request  
        try:  
            raw = urllib.request.urlopen(GLASS_URL, timeout=30).read()  
            digest = hashlib.sha256(raw).hexdigest()  
        except Exception:  
            raw = zlib.decompress(base64.b85decode(DATA))  
            digest = hashlib.sha256(raw).hexdigest()  
  
    reader = csv.reader(io.StringIO(raw.decode('utf-8-sig')))  
    rows = [r for r in reader if r and not r[0].strip().startswith('#') and not  
r[0].strip().lower().startswith('id')]  
    if not rows:  
        raise ValueError('Пустой CSV')  
  
    X = np.array([[float(r[1 + i]) for i in range(9)] for r in rows], dtype=float)  
    y_raw = np.array([int(float(r[10])) for r in rows], dtype=int)  
  
    present = sorted(np.unique(y_raw).tolist())  
    remap = {orig: new for new, orig in enumerate(present)}  
    y = np.array([remap[v] for v in y_raw], dtype=int)  
    class_names_local = [GLASS_NAMES.get(orig, f'class_{orig}') for orig in present]  
  
    if X.shape[1] != 9 or not np.isfinite(X).all():  
        raise ValueError('Ожидается 9 числовых признаков без пропусков')  
  
    return X, y, digest, class_names_local  
  
def models(n_classes):  
    ada_args = dict(estimator=DecisionTreeClassifier(max_depth=1, random_state=42),  
                    n_estimators=200, learning_rate=.5, random_state=42)  
    import inspect  
    if 'algorithm' in inspect.signature(AdaBoostClassifier).parameters:  
        ada_args['algorithm'] = 'SAMME'  
    return [DecisionTreeClassifier(max_depth=5, random_state=42),  
            RandomForestClassifier(n_estimators=300, max_features='sqrt',  
                                  random_state=42, n_jobs=1),  
            AdaBoostClassifier(**ada_args),  
            CatBoostClassifier(iterations=300, depth=4, learning_rate=.05,  
                               loss_function='MultiClass', random_seed=42,  
                               thread_count=1, verbose=False, allow_writing_files=False),  
            XGBClassifier(n_estimators=300, max_depth=3, learning_rate=.05,  
                           objective='multi:softprob', num_class=n_classes,  
eval_metric='mlogloss',  
                           tree_method='hist', subsample=1., colsample_bytree=1.,  
                           random_state=42, n_jobs=1)]
```

```

def split(y):
    return train_test_split(np.arange(len(y)), test_size=.3, stratify=y, random_state=42)

def self_test():
    X, y, _, cls = load_data()
    Xs = StandardScaler().fit_transform(X)
    tr, te = split(y)
    assert len(tr) + len(te) == len(y) and not set(tr) & set(te)
    assert sorted(np.r_[tr, te]) == list(range(len(y)))
    assert len(cls) == len(np.unique(y))
    folds = list(StratifiedKFold(5, shuffle=True, random_state=42).split(Xs[tr], y[tr]))
    held = []
    for a, b in folds:
        assert not set(tr[a]) & set(te) and not set(a) & set(b)
        held.extend(b)
    assert sorted(held) == list(range(len(tr)))
    for m in models(len(cls)):
        a = clone(m).fit(Xs[tr], y[tr]); b = clone(m).fit(Xs[tr], y[tr])
        pred = np.asarray(a.predict(Xs[te])).reshape(-1).astype(int)
        assert np.array_equal(pred, np.asarray(b.predict(Xs[te])).reshape(-1))
        prob = a.predict_proba(Xs[te]); assert prob.shape == (len(te), len(cls))
        assert np.isfinite(prob).all() and np.allclose(prob.sum(axis=1), 1, atol=1e-6)
        cm = confusion_matrix(y[te], pred, labels=list(range(len(cls))))
        assert cm.sum() == len(te) and np.isclose(np.trace(cm) / len(te), accuracy_score(y[te],
pred))
        print('Проверки CSV, разбиения, CV, вероятностей, метрик и повторяемости пройдены.')

def main():
    ap = argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--csv', type=Path)
    ap.add_argument('--out', type=Path, default=Path(__file__).resolve().parent / 'results_lab5')
    ap.add_argument('--no-show', action='store_true')
    ap.add_argument('--self-test', action='store_true')
    args = ap.parse_args()
    if args.self_test:
        self_test()
        return

    import matplotlib
    if args.no_show:
        matplotlib.use('Agg')
    import matplotlib.pyplot as plt

    X, y, digest, class_names_local = load_data(args.csv)
    n_classes = len(class_names_local)

    scaler = StandardScaler()
    Xs = scaler.fit_transform(X)

    tr, te = split(y)
    out = args.out
    out.mkdir(parents=True, exist_ok=True)

    folds = list(StratifiedKFold(5, shuffle=True, random_state=42).split(Xs[tr], y[tr]))
    results = []
    fitted = []

    for name, model in zip(NAMES, models(n_classes)):
        scores = []
        for a, b in folds:
            fold = clone(model).fit(Xs[tr[a]], y[tr[a]])
            scores.append(accuracy_score(y[tr[b]], np.asarray(fold.predict(Xs[tr[b]])).reshape(-
1)))

        start = time.perf_counter()
        model.fit(Xs[tr], y[tr])
        elapsed = time.perf_counter() - start

```

```

pred = np.asarray(model.predict(Xs[te])).reshape(-1).astype(int)
train_pred = np.asarray(model.predict(Xs[tr])).reshape(-1).astype(int)
cm = confusion_matrix(y[te], pred, labels=list(range(n_classes)))
rec = dict(name=name,
           accuracy=accuracy_score(y[te], pred),
           train_accuracy=accuracy_score(y[tr], train_pred),
           macro_f1=f1_score(y[te], pred, average='macro'),
           correct=int(np.trace(cm)),
           errors=int((pred != y[te]).sum()),
           cm=cm.tolist(),
           cv_scores=scores,
           cv_mean=float(np.mean(scores)),
           cv_std=float(np.std(scores, ddof=1)),
           seconds=elapsed,
           predictions=pred.tolist(),
           report=classification_report(y[te], pred, target_names=class_names_local,
                                       output_dict=True, zero_division=0))

results.append(rec)
fitted.append(model)
print(f'{name:15} test accuracy={rec["accuracy"]:.4f}; F1={rec["macro_f1"]:.4f};
CV={rec["cv_mean"]:.4f} ± {rec["cv_std"]:.4f}', flush=True)

best_idx = int(np.argmax([r['accuracy'] for r in results]))
best = results[best_idx]
print(f'\nЛучшая модель: {best["name"]} (accuracy={best["accuracy"]:.4f})')
print('Recall по классам для лучшей модели:')
worst = None
worst_recall = 1.0
for cname in class_names_local:
    rr = best['report'].get(cname, {})
    r = rr.get('recall', 0.0)
    s = rr.get('support', 0)
    print(f' {cname}: recall={r:.4f}, support={int(s)}')
    if s > 0 and r < worst_recall:
        worst_recall = r
        worst = cname
print(f'\nХуже всего определяется: {worst} (recall={worst_recall:.4f})')

summary = dict(source=str(args.csv) if args.csv else GLASS_URL,
              sha256=digest,
              versions=dict(python=platform.python_version(), numpy=np.__version__,
                           sklearn=sklearn.__version__, catboost=catboost.__version__,
                           xgboost=xgboost.__version__),
              train_indices=tr.tolist(), test_indices=te.tolist(),
              class_names=class_names_local,
              duplicate_extra_rows=int(len(X) - len(np.unique(np.c_[X, y], axis=0))),
              statistics=dict(min=X.min(0).tolist(), max=X.max(0).tolist(),
                              mean=X.mean(0).tolist(), std=X.std(0, ddof=1).tolist()),
              models=results,
              tree_depth=int(fitted[0].get_depth()),
              tree_leaves=int(fitted[0].get_n_leaves()),
              tree_importances=fitted[0].feature_importances_.tolist())

(out / 'results.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding='utf-8')

with (out / 'metrics.csv').open('w', encoding='utf-8-sig', newline='') as fp:
    fields = ['name', 'accuracy', 'train_accuracy', 'macro_f1', 'correct', 'errors',
'cv_mean', 'cv_std', 'seconds']
    w = csv.DictWriter(fp, fieldnames=fields)
    w.writeheader()
    w.writerows([{'k': r[k] for k in fields} for r in results])

with (out / 'predictions.csv').open('w', encoding='utf-8-sig', newline='') as fp:
    w = csv.writer(fp)
    w.writerow(['csv_line', 'true'] + NAMES)
    for j, idx in enumerate(te):
        w.writerow([int(idx) + 1, class_names_local[y[idx]]] +
                    [class_names_local[r['predictions'][j]] for r in results])

```

```

(out / 'tree_rules.txt').write_text(export_text(fitted[0], feature_names=FEATURES),
encoding='utf-8')

plt.rcParams.update({'font.family': 'DejaVu Sans', 'font.size': 10})

def save(fig, name):
    fig.savefig(out / name, dpi=190, bbox_inches='tight')

fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), layout='constrained')
for ax, (a, b) in zip(axes, [(0, 3), (1, 6)]):
    for c, label in enumerate(class_names_local):
        ax.scatter(Xs[y == c, a], Xs[y == c, b], label=label, s=22, alpha=.75)
    ax.set(xlabel=FEATURES[a], ylabel=FEATURES[b])
    ax.grid(alpha=.2)
axes[0].legend(fontsize=7)
save(fig, 'data.png')

fig, ax = plt.subplots(figsize=(9, 3.5), layout='constrained')
xx = np.arange(5)
ax.bar(xx - .18, [r['accuracy'] for r in results], .36, label='Test')
ax.bar(xx + .18, [r['cv_mean'] for r in results], .36,
       yerr=[r['cv_std'] for r in results], capsize=3, label='CV на train ± SD')
ax.set(xticks=xx, xticklabels=NAMES, ylim=(0, 1.12), ylabel='Accuracy')
ax.legend()
ax.grid(axis='y', alpha=.2)
for i, r in enumerate(results):
    ax.text(i - .18, .03, f'{r["correct"]}/{len(te)}', ha='center', color='white')
save(fig, 'comparison.png')

fig, axes = plt.subplots(2, 3, figsize=(10, 6), layout='constrained')
for ax, r in zip(axes.flat, results):
    cm = np.array(r['cm'])
    ax.imshow(cm, cmap='Blues')
    ax.set(title=r['name'], xticks=range(n_classes), yticks=range(n_classes),
          xticklabels=[c[:6] for c in class_names_local],
          yticklabels=[c[:6] for c in class_names_local],
          xlabel='Прогноз', ylabel='Истинный класс')
    for (i, j), v in np.ndenumerate(cm):
        ax.text(j, i, str(v), ha='center', va='center',
              color='white' if v > cm.max() / 2 else 'black')
axes.flat[-1].axis('off')
save(fig, 'confusion.png')

fig, ax = plt.subplots(figsize=(13, 7), layout='constrained')
plot_tree(fitted[0], feature_names=FEATURES, class_names=class_names_local,
          filled=True, rounded=True, precision=2, fontsize=9, ax=ax)
save(fig, 'tree.png')

print('Результаты:', out.resolve())
if not args.no_show:
    plt.show()
plt.close('all')

if __name__ == '__main__':
    main()

```

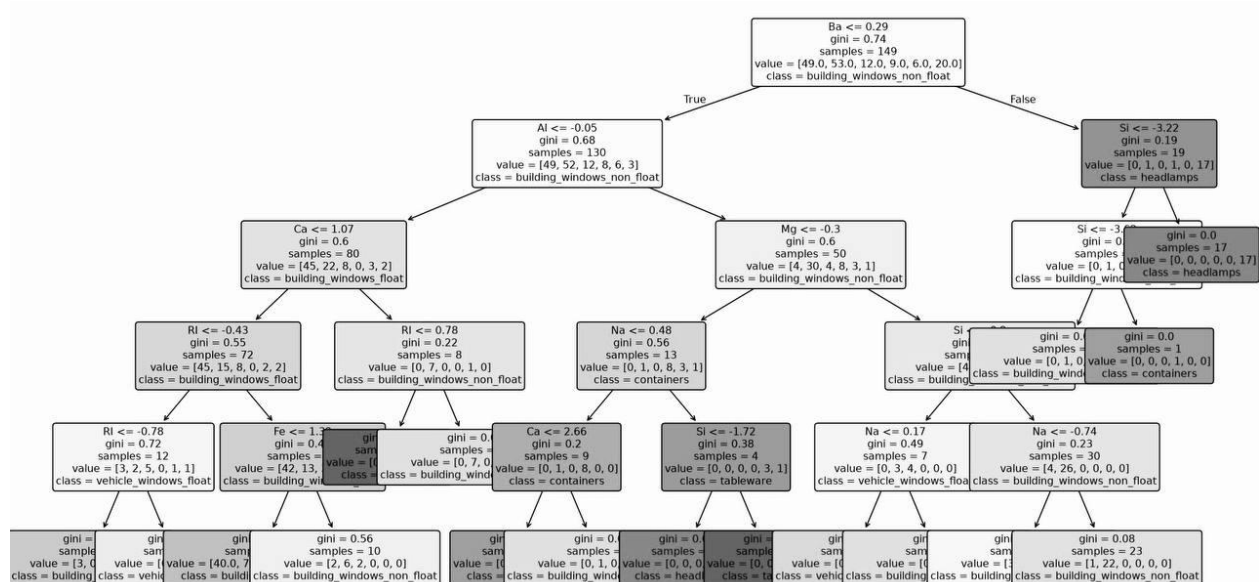
Результат программы:

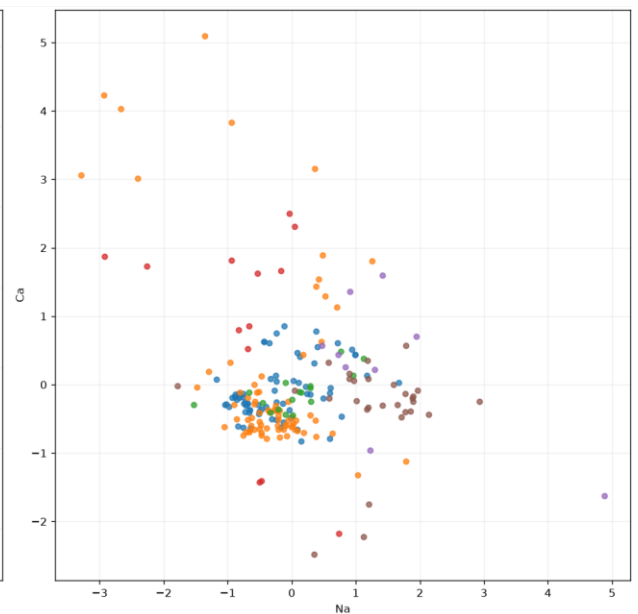
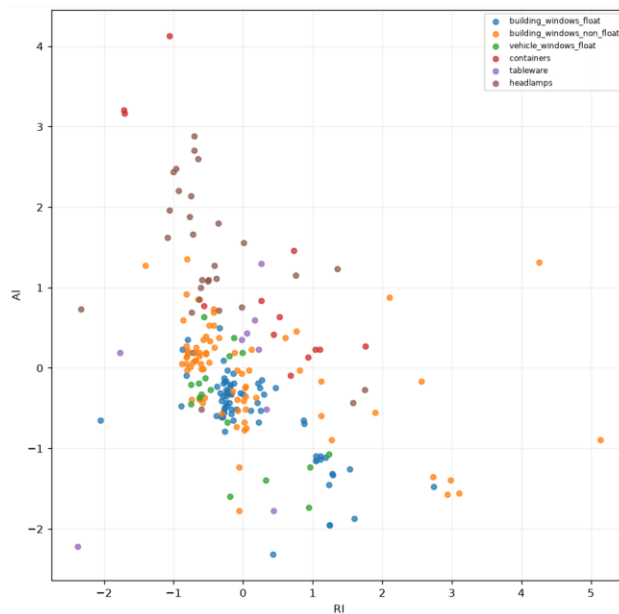
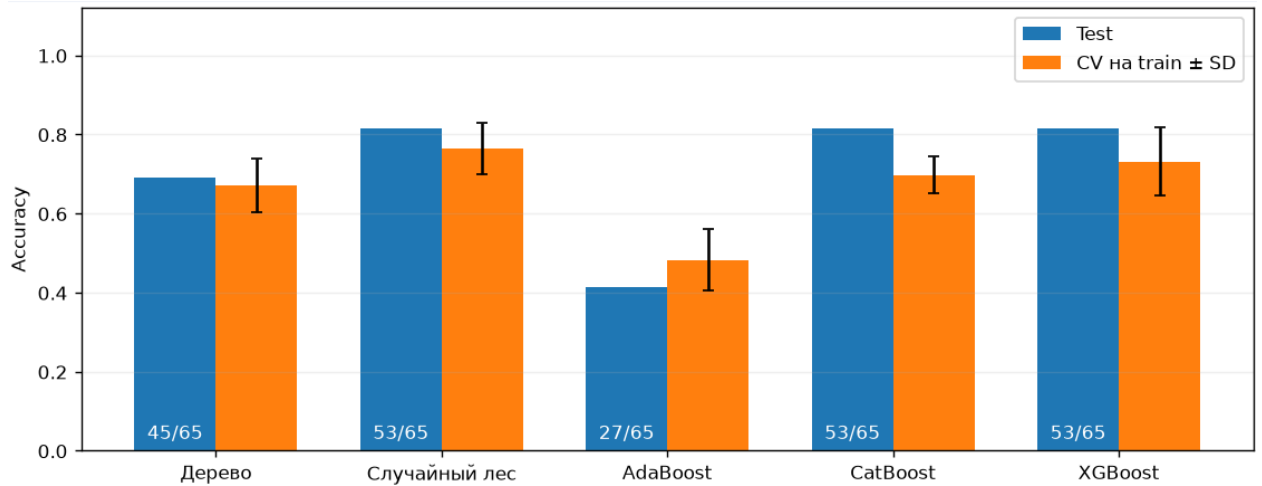
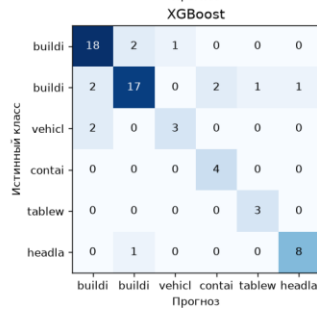
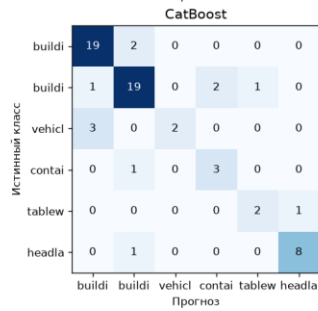
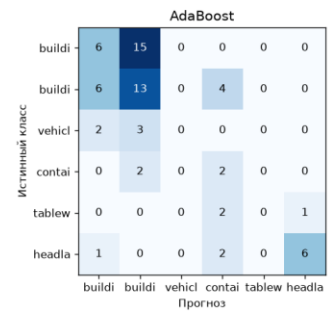
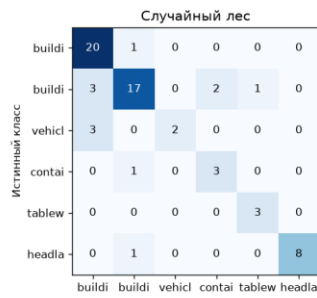
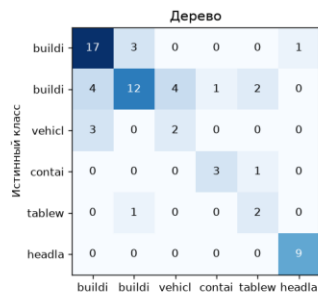
Дерево test accuracy=0.6923; F1=0.6553; CV=0.6710 ± 0.0688

Случайный лес test accuracy=0.8154; F1=0.7797; CV=0.7646 ± 0.0652

AdaBoost test accuracy=0.4154; F1=0.3056; CV=0.4830 ± 0.0784

CatBoost test accuracy=0.8154; F1=0.7472; CV=0.6979 ± 0.0474





В ходе работы на датасете Glass Identification были обучены пять моделей: одиночное дерево решений, случайный лес, AdaBoost, CatBoost и XGBoost. Лучшую точность показали три модели одновременно, Random Forest, CatBoost и XGBoost, все с accuracy =

0.8154, однако по макро-F1 вперёд вышел XGBoost (0.8068), то есть он наиболее равномерно определяет все шесть классов. Random Forest при этом оказался стабильнее по кросс-валидации (0.7646 ± 0.0652), а CatBoost наиболее устойчив (наименьшее стандартное отклонение 0.0474). Одиночное дерево ожидаемо уступило ансамблям (0.6923), а AdaBoost показал наихудший результат (0.4154), его базовый классификатор-пень слишком слаб для шестиклассовой задачи с сильным дисбалансом. Хуже всего все модели определяют класс `vehicle_windows_float` ($\text{recall} = 0.4$): в датасете всего 17 его представителей, в тестовую выборку попало 5 объектов, и по химическому составу он близок к `building_windows_float`, поэтому модель часто их путает. Аналогичная проблема у малочисленных классов `containers` (13 объектов) и `tableware` (9 объектов), их recall нестабилен из-за малого `support`. Основная причина ошибок, сильный дисбаланс классов в датасете Glass, а не слабость алгоритмов: ансамблевые методы с ним справляются заметно лучше одиночного дерева и AdaBoost.

Вывод: на практике сравнила работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.